

DOM परिचय



You said:

dom introductions

2/2

ChatGPT said:

DOM (Document Object Model) – Introduction 🌐

DOM का पूरा नाम **Document Object Model** है। यह एक **programming interface** है जो **HTML और XML documents** को **JavaScript** के माध्यम से **access** और **manipulate** करने की सुविधा देता है।

सरल शब्दों में:

👉 **DOM = HTML page का tree structure** जिसे JavaScript बदल सकती है।

जब browser किसी **HTML page** को load करता है, तो वह उस page को **DOM tree** में convert कर देता है।

1 DOM क्या है? (Easy Definition)

DOM एक object structure है जो HTML elements को objects की तरह represent करता है।

इसका मतलब:

- हर **HTML tag** एक **object** बन जाता है
- JavaScript उन objects को **read, change, add, delete** कर सकती है

Example HTML:

```
HTML
<html>
<head>
<title>My Page</title>
</head>

<body>
<h1>Hello</h1>
<p>This is paragraph</p>
</body>
</html>
```

Browser इसे DOM tree में बदल देता है.

2 DOM Tree Structure 🌳

DOM एक **tree structure** की तरह होता है।

```
Document
  |
  html
    /  \
  head  body
    |    |
  title  h1
         |
         p
```

यहाँ:

- **Document** → root object
- **html** → parent node
- **head / body** → child nodes
- **title / h1 / p** → elements

3 DOM में Nodes क्या होते हैं?

DOM में हर चीज **Node** कहलाती है।

Types of nodes:

1 Document Node

```
document
```

यह पूरे webpage को represent करता है।

2 Element Node

HTML tags

```
<h1>  
<p>  
<div>
```

3 Text Node

Element के अंदर का text

```
<p>Hello World</p>
```

4 Attribute Node

Element की properties

```

```

4 JavaScript DOM क्यों use करता है?

JavaScript DOM की मदद से webpage को **dynamic** बनाता है।

आप यह सब कर सकते हो:

- ✓ Text change
- ✓ Style change
- ✓ Element add/remove
- ✓ Event handling
- ✓ Form validation

5 DOM Example

HTML:

```
HTML
<h1 id="title">Hello</h1>
```

JavaScript:

```
JavaScript
let heading = document.getElementById("title");

heading.innerText = "Hello JavaScript";
```

Output:

```
Hello JavaScript
```

JavaScript ने **HTML content change** कर दिया।

6 DOM Manipulation

DOM manipulation का मतलब:

👉 Webpage के elements को **change करना**

Examples:

Operation	Example
Select element	getElementById()
Change text	innerText
Change HTML	innerHTML
Change style	style
Add element	appendChild()
Remove element	remove()

7 DOM Selectors

JavaScript में elements select करने के methods:

1. getElementById()

```
JavaScript  
document.getElementById("id")
```

2. getElementsByClassName()

```
JavaScript  
document.getElementsByClassName("class")
```

3. getElementsByTagName()

```
JavaScript  
document.getElementsByTagName("p")
```

4. querySelector()

```
JavaScript  
document.querySelector(".class")
```

5. querySelectorAll()

```
JavaScript  
document.querySelectorAll("p")
```

8 Real Life Example

HTML

```
HTML  
<button onclick="changeText()">Click</button>  
<p id="demo">Hello</p>
```

JavaScript

```
JavaScript
function changeText() {
  document.getElementById("demo").innerText="Welcome";
}
```

Button click → Text change

9 DOM का Flow

```
HTML Page
  ↓
Browser loads page
  ↓
DOM Tree create
  ↓
JavaScript access DOM
  ↓
Modify elements
```

10 DOM vs HTML

HTML	DOM
Static	Dynamic
Structure	Object representation
Written by developer	Created by browser

✓ One line interview definition

DOM is a programming interface that represents HTML documents as a tree of objects so JavaScript can manipulate them.

✓ अगर आप चाहें तो मैं आगे ये भी समझा सकता हूँ:

- **DOM Manipulation (Complete)**
- **DOM Events (click, submit, input)**
- **DOM Interview Questions** 🔥
- **DOM Practice Questions**

- **Mini Project using DOM (important for MERN)**

बस बोलो “**DOM full roadmap**” 🚀

You said:

dom introductions in english

ChatGPT said:

DOM Introduction (Document Object Model) 🌐

The **DOM (Document Object Model)** is a **programming interface for HTML and XML documents**. It represents a web page so that programs (like **JavaScript**) can **access, change, and manipulate the content, structure, and style of the document**.

In simple words:

👉 **DOM = A tree-like structure of an HTML page that JavaScript can control.**

When a **browser loads an HTML page**, it converts the HTML document into a **DOM tree**.

1. What is DOM? (Simple Definition)

The **DOM represents an HTML document as a collection of objects**.

Each part of the webpage becomes an object that JavaScript can access and modify.

For example:

HTML code:

```
HTML
<html>
<head>
<title>My Page</title>
</head>

<body>
<h1>Hello</h1>
<p>This is a paragraph</p>
</body>
</html>
```

The browser converts this into a **DOM structure**.

2. DOM Tree Structure

The DOM represents the HTML document as a **tree structure**.

```
Document
  |
  html
  /  \
head  body
 |      |
title  h1
       |
       p
```

Explanation:

- **Document** → Root of the page
- **html** → Parent element
- **head & body** → Child elements
- **title, h1, p** → Other elements

3. What are Nodes in DOM?

In the DOM, everything is called a **node**.

Types of nodes:

1. Document Node

Represents the entire HTML document.

```
document
```

2. Element Node

Represents HTML elements.

Examples:

```
<h1>  
<p>  
<div>
```

3. Text Node

Represents the text inside elements.

Example:

```
HTML  
<p>Hello World</p>
```

Here **"Hello World"** is a text node.

4. Attribute Node

Represents attributes of elements.

Example:

```
HTML  

```

Here **src** is an attribute node.

4. Why Do We Use DOM?

JavaScript uses the DOM to make web pages **dynamic and interactive**.

With DOM we can:

- ✓ Change text content
- ✓ Change styles (CSS)
- ✓ Add new elements
- ✓ Remove elements
- ✓ Handle user events (click, input, submit)

5. DOM Example

HTML:

```
HTML
<h1 id="title">Hello</h1>
```

JavaScript:

```
JavaScript
let heading = document.getElementById("title");

heading.innerText = "Hello JavaScript";
```

Output:

```
Hello JavaScript
```

JavaScript **changes the text using DOM.**

6. DOM Manipulation

DOM Manipulation means **changing or controlling HTML elements using JavaScript.**

Examples:

- Changing text
- Changing styles
- Adding new elements
- Removing elements
- Handling events

7. DOM Selection Methods

JavaScript provides methods to select elements.

getElementById()

```
JavaScript
document.getElementById("id")
```

getElementsByClassName()

```
JavaScript
document.getElementsByClassName("class")
```

getElementsByName()

```
JavaScript
document.getElementsByName("p")
```

querySelector()

```
JavaScript
document.querySelector(".class")
```

querySelectorAll()

```
JavaScript
document.querySelectorAll("p")
```

8. Real Example

HTML:

```
HTML
<button onclick="changeText()">Click</button>
<p id="demo">Hello</p>
```

JavaScript:

```
JavaScript
function changeText() {
    document.getElementById("demo").innerText = "Welcome";
}
```

When the button is clicked, the **text changes**.

9. DOM Flow

```
HTML Page
↓
Browser loads HTML
↓
DOM Tree is created
↓
JavaScript accesses DOM
↓
Webpage changes dynamically
```

10. One-Line Interview Definition

DOM (Document Object Model) is a programming interface that represents HTML documents as a tree of objects so that JavaScript can access and manipulate them.

✅ If you want, I can also explain next:

- **DOM Manipulation (Complete Guide)**
- **DOM Events (click, input, submit)**
- **DOM Interview Questions**
- **DOM Practice Questions**
- **Mini Project using DOM**

Just tell me **“Teach DOM completely”** and I’ll explain it step-by-step. 🚀

You said:

what is dom

ChatGPT said:

What is DOM? 🌐

DOM (Document Object Model) is a programming interface for HTML and XML documents that allows JavaScript to access, modify, and control the content, structure, and style of a web page.

Simple Definition

👉 **DOM is a tree-like structure created by the browser from an HTML document, which JavaScript can use to interact with the webpage.**

Example

HTML

```
HTML
<h1 id="title">Hello</h1>
```

JavaScript

```
JavaScript
let heading = document.getElementById("title");
heading.innerText = "Hello JavaScript";
```

Here, JavaScript uses the **DOM** to **change the text of the HTML element**.

How DOM Works

1. A browser loads an **HTML page**.
2. The browser converts the HTML into a **DOM tree**.
3. **JavaScript accesses the DOM** using methods like `getElementById()`.
4. JavaScript can **change the content, style, or structure** of the page.

DOM Tree Example

```
Document
  |
  html
  /  \
head  body
      |
      h1
      |
      p
```

Each HTML element becomes a **node (object)** in the DOM.

Why DOM is Used

Using DOM, JavaScript can:

- Change text or content
- Change CSS styles
- Add new HTML elements
- Remove elements
- Respond to user actions (click, input, submit)

✅ **One-line interview answer:**

DOM is a programming interface that represents an HTML document as a tree of objects so that JavaScript can access and manipulate the webpage.

You said:

selecting element by id

ChatGPT said:

Selecting Element by ID in DOM

In **DOM**, we can select an HTML element using its **id attribute** with the JavaScript method:

```
document.getElementById()
```

Definition

getElementById() is a DOM method used to **select an HTML element using its unique id**.

Since an **id is always unique**, this method returns **only one element**.

Syntax

```
JavaScript  
document.getElementById("idName");
```

- **document** → represents the whole HTML page

- `getElementById()` → DOM method
- `"idName"` → the id of the element

Example

HTML

```
HTML
<h1 id="title">Hello World</h1>
```

JavaScript

```
JavaScript
let element = document.getElementById("title");

console.log(element);
```

Output:

```
<h1 id="title">Hello World</h1>
```

JavaScript selects the **h1 element**.

Changing Text Using ID

HTML

```
HTML
<p id="demo">Hello</p>
```

JavaScript

```
JavaScript
let para = document.getElementById("demo");

para.innerText = "Welcome to JavaScript";
```

Output:

```
Welcome to JavaScript
```

The text changes using DOM.

Changing Style Using ID

```
JavaScript
let heading = document.getElementById("title");

heading.style.color = "red";
heading.style.fontSize = "40px";
```

This changes the **color and size of the text**.

Button Example

HTML

```
HTML
<button onclick="changeText()">Click</button>

<p id="demo">Hello</p>
```

JavaScript

```
JavaScript
function changeText() {
    document.getElementById("demo").innerText = "You clicked the
button!";
}
```

When the button is clicked, the **text changes**.

Important Points

- 1 `getElementById()` selects **only one element**.
- 2 The **id must be unique** in the HTML page.
- 3 Always write the **id name inside quotes**.

Interview Question

Q: What is `getElementById()` in JavaScript?

Answer:

`getElementById()` is a DOM method used to select an HTML element by its id so that JavaScript can access or modify it.

If you want, I can also explain the **next important DOM topics**:

- Selecting Elements by **Class Name**
- Selecting Elements by **Tag Name**
- **querySelector & querySelectorAll**
- **DOM Manipulation**
- **DOM Events**

Just tell me “**next DOM topic**” 🚀

You said:

selecting element by class name

ChatGPT said:

Selecting Element by Class Name in DOM

In JavaScript DOM, we can select elements using their **class name** with the method:

```
document.getElementsByClassName()
```

Definition

`getElementsByClassName()` is a DOM method used to **select all HTML elements that have the same class name**.

Unlike **id**, a class can be used **multiple times**, so this method returns a **collection of elements (HTMLCollection)**.

Syntax

```
JavaScript
document.getElementsByClassName("className");
```

- **document** → represents the whole HTML page
- **getElementsByClassName()** → DOM method
- **"className"** → name of the class

Example

HTML

```
HTML
<p class="text">Hello</p>
<p class="text">Welcome</p>
<p class="text">JavaScript</p>
```

JavaScript

```
JavaScript
let elements = document.getElementsByClassName("text");

console.log(elements);
```

Output:

```
HTMLCollection(3)
```

It selects **all three paragraph elements**.

Accessing Specific Element

Since it returns a **collection**, we use **index numbers**.

```
JavaScript
let elements = document.getElementsByClassName("text");

console.log(elements[0]);
console.log(elements[1]);
```

Index starts from **0**.

Changing Text

```
JavaScript
let elements = document.getElementsByClassName("text");

elements[0].innerText = "Hello World";
```

This changes the **first paragraph text**.

Changing Style

```
JavaScript
let elements = document.getElementsByClassName("text");

elements[1].style.color = "blue";
```

This changes the **color of the second paragraph**.

Example with Loop

If we want to change **all elements**, we use a loop.

HTML

```
HTML
<p class="item">Item 1</p>
<p class="item">Item 2</p>
<p class="item">Item 3</p>
```

JavaScript

```
JavaScript
let items = document.getElementsByClassName("item");

for(let i = 0; i < items.length; i++){
    items[i].style.color = "red";
}
```

All items become **red**.

Important Points

- 1 Class name can be **used multiple times**.
- 2 It returns an **HTMLCollection (list of elements)**.
- 3 Use **index** to access specific elements.
- 4 Use **loop** to apply changes to all elements.

Interview Question

Q: What does `getElementsByClassName()` return?

Answer:

`getElementsByClassName()` returns an **HTMLCollection of elements that have the specified class name**.

If you want, I can also explain the **next DOM selectors (very important)**:

- **Selecting Elements by Tag Name**
- **`querySelector()`**
- **`querySelectorAll()`**
- **Difference between all DOM selectors (Interview Important).**

You said:

selecting element by tag name

ChatGPT said:

Selecting Element by Tag Name in DOM

In JavaScript DOM, we can select HTML elements using their **tag name** with the method:

```
document.getElementsByTagName()
```

Definition

getElementsByTagName() is a DOM method used to **select all HTML elements that have the same tag name**.

It returns a **collection of elements (HTMLCollection)**.

Syntax

```
JavaScript  
document.getElementsByTagName("tagName");
```

- **document** → represents the whole HTML page
- **getElementsByTagName()** → DOM method
- **tagName** → HTML tag (like **p**, **h1**, **div**)

Example

HTML

```
HTML  
<p>Hello</p>  
<p>Welcome</p>  
<p>JavaScript</p>
```

JavaScript

```
JavaScript  
let elements = document.getElementsByTagName("p");  
  
console.log(elements);
```

Output:

```
HTMLCollection(3)
```

It selects **all paragraph elements**.

Accessing Elements

Since it returns a **collection**, we access elements using **index numbers**.

```
JavaScript
let elements = document.getElementsByTagName("p");

console.log(elements[0]);
console.log(elements[1]);
```

Index starts from **0**.

Changing Text

```
JavaScript
let elements = document.getElementsByTagName("p");

elements[0].innerText = "Hello World";
```

This changes the **first paragraph text**.

Changing Style

```
JavaScript
let elements = document.getElementsByTagName("p");

elements[1].style.color = "blue";
```

This changes the **color of the second paragraph**.

Example Using Loop

If we want to change **all elements**, we use a loop.

HTML

```
HTML
<h2>Heading 1</h2>
<h2>Heading 2</h2>
<h2>Heading 3</h2>
```

JavaScript

```
JavaScript
let headings = document.getElementsByTagName("h2");

for(let i = 0; i < headings.length; i++){
    headings[i].style.color = "red";
}
```

All headings become **red**.

Important Points

- 1 It selects **all elements with the same tag name**.
- 2 It returns an **HTMLCollection**.
- 3 We use **index numbers** to access elements.
- 4 We often use a **loop** to modify multiple elements.

Interview Question

Q: What does `getElementsByTagName()` return?

Answer:

`getElementsByTagName()` returns an **HTMLCollection** containing all elements with the specified tag name.

If you want, I can also explain the **most important DOM selectors used in modern JavaScript**:

- `querySelector()`
- `querySelectorAll()`
- **Difference between all DOM selectors (Very Important for Interviews).**

You said:

query selectors all concept

ChatGPT said:

Query Selectors in DOM (Complete Concept)

In modern JavaScript, **query selectors** are the most commonly used methods to select HTML elements.

There are **two main query selector methods**:

1. `querySelector()`
2. `querySelectorAll()`

These methods use **CSS selectors** to select elements.

1. querySelector()

Definition

`querySelector()` is used to **select the first element that matches a CSS selector**.

Syntax

```
JavaScript  
document.querySelector("CSS selector");
```

It returns **only the first matching element**.

Example 1: Selecting by Tag

HTML

```
HTML  
<p>Hello</p>  
<p>Welcome</p>
```

JavaScript

```
JavaScript
let para = document.querySelector("p");
console.log(para);
```

It selects **only the first** `<p>` element.

Example 2: Selecting by ID

HTML

```
HTML
<h1 id="title">Hello</h1>
```

JavaScript

```
JavaScript
let heading = document.querySelector("#title");
```

`#` is used for **id**.

Example 3: Selecting by Class

HTML

```
HTML
<p class="text">Hello</p>
<p class="text">Welcome</p>
```

JavaScript

```
JavaScript
let element = document.querySelector(".text");
```

`.` is used for **class**.

This selects **only the first element with class "text"**.

2. querySelectorAll()

Definition

`querySelectorAll()` selects **all elements that match a CSS selector**.

It returns a **NodeList (list of elements)**.

Syntax

```
JavaScript
document.querySelectorAll("CSS selector");
```

Example

HTML

```
HTML
<p class="text">Hello</p>
<p class="text">Welcome</p>
<p class="text">JavaScript</p>
```

JavaScript

```
JavaScript
let elements = document.querySelectorAll(".text");

console.log(elements);
```

Output:

```
NodeList(3)
```

It selects **all elements with class "text"**.

Accessing Elements

```
JavaScript
let elements = document.querySelectorAll(".text");

console.log(elements[0]);
console.log(elements[1]);
```

Using Loop

```
JavaScript
let elements = document.querySelectorAll(".text");

elements.forEach(function(el) {
    el.style.color = "red";
});
```

All elements become **red**.

CSS Selectors Used with QuerySelector

1. Tag selector

```
JavaScript
document.querySelector("p");
```

2. ID selector

```
JavaScript
document.querySelector("#title");
```

3. Class selector

```
JavaScript
document.querySelector(".text");
```

4. Nested selector

```
JavaScript
document.querySelector("div p");
```

Selects **p inside div**.

Example (Real DOM Example)

HTML

```
HTML
<div>
  <p class="text">Hello</p>
  <p class="text">Welcome</p>
</div>
```

JavaScript

```
JavaScript
let elements = document.querySelectorAll("div .text");

elements.forEach(function(el) {
  el.style.color = "blue";
});
```

Difference Between querySelector and querySelectorAll

Method	Result
querySelector()	First matching element
querySelectorAll()	All matching elements
Return type	Element
Return type	NodeList

Important Points

1. Query selectors use **CSS selector syntax**.

2. `querySelector()` returns **only the first element**.
3. `querySelectorAll()` returns **all elements**.
4. `querySelectorAll()` returns a **NodeList**.

Interview Question

Q: What is the difference between `querySelector()` and `querySelectorAll()`?

Answer:

- `querySelector()` selects the **first matching element**.
- `querySelectorAll()` selects **all matching elements** and returns a **NodeList**.

If you want, I can also explain the **next important DOM topics**:

- **DOM Manipulation (Very Important)**
- **innerText vs innerHTML vs textContent**
- **getAttribute / setAttribute**
- **classList (add, remove, toggle)**
- **DOM Events**

Just say **“teach DOM manipulation”** and I will explain it step-by-step. 🚀

You said:

setting content objects

ChatGPT said:

Setting Content in DOM (Objects)

In the DOM, we can **change or set the content of HTML elements using JavaScript**.

This is called **setting content**.

JavaScript treats HTML elements as **objects**, so we can modify their properties like:

- `innerText`

- `textContent`
- `innerHTML`

1. innerText

Definition

`innerText` is used to **set or change the visible text inside an element**.

Example

HTML

```
HTML
<p id="demo">Hello</p>
```

JavaScript

```
JavaScript
let element = document.getElementById("demo");

element.innerText = "Welcome to JavaScript";
```

Output

```
Welcome to JavaScript
```

It changes the text inside the paragraph.

2. textContent

Definition

`textContent` is used to **set or get all text inside an element**, including hidden text.

Example

```
JavaScript
let element = document.getElementById("demo");

element.textContent = "Learning DOM";
```

It changes the text just like **innerText**.

3. innerHTML

Definition

innerHTML is used to **set or get HTML content inside an element**.

It can include **HTML tags**.

Example

HTML

```
HTML
<div id="box"></div>
```

JavaScript

```
JavaScript
let box = document.getElementById("box");

box.innerHTML = "<h2>Hello JavaScript</h2>";
```

Output

```
Hello JavaScript
```

A new HTML element is created inside the div.

Example Combining Everything

HTML

```
HTML
<p id="text">Hello</p>
<div id="container"></div>
```

JavaScript

```
JavaScript
let text = document.getElementById("text");
let container = document.getElementById("container");

text.innerText = "Welcome";

container.innerHTML = "<h3>Learning DOM</h3>";
```

Difference Between innerText, textContent, innerHTML

Property	Use
innerText	Gets/sets visible text
textContent	Gets/sets all text
innerHTML	Gets/sets HTML content

Important Points

1. **innerText** changes **only text**.
2. **innerHTML** can insert **HTML tags**.
3. **textContent** returns **all text inside the element**.

Interview Question

Q: What is the difference between innerText and innerHTML?

Answer:

- **innerText** changes only the **text content** of an element.

- **innerHTML** can change **HTML structure and text inside the element**.

If you want, I can also explain the **next important DOM topics**:

- **Setting Attributes (getAttribute / setAttribute)**
- **classList (add, remove, toggle)**
- **DOM Manipulation**
- **DOM Events (click, submit, input)**
- **Create and Remove Elements in DOM**

Just tell me “**next DOM concept**” 🚀

You said:

manipulating attributes

ChatGPT said:

Manipulating Attributes in DOM

In JavaScript DOM, **attributes of HTML elements** (like **id**, **class**, **src**, **href**) can be **read, changed, added, or removed**.

This process is called **manipulating attributes**.

JavaScript provides these main methods:

1. **getAttribute()**
2. **setAttribute()**
3. **removeAttribute()**

1. getAttribute()

Definition

getAttribute() is used to **get the value of an attribute** from an HTML element.

Syntax

```
JavaScript  
element.getAttribute("attributeName");
```

Example

HTML

```
HTML

```

JavaScript

```
JavaScript
let img = document.getElementById("image");

let value = img.getAttribute("src");

console.log(value);
```

Output

```
photo.jpg
```

2. setAttribute()

Definition

setAttribute() is used to **add a new attribute or change an existing attribute**.

Syntax

```
JavaScript
element.setAttribute("attributeName", "value");
```

Example

HTML

```
HTML
<a id="link">Visit Website</a>
```

JavaScript

```
JavaScript
let link = document.getElementById("link");

link.setAttribute("href", "https://google.com");
```

Now the link will open **Google**.

3. removeAttribute()

Definition

`removeAttribute()` is used to **remove an attribute from an element**.

Syntax

```
JavaScript
element.removeAttribute("attributeName");
```

Example

HTML

```
HTML
<button id="btn" disabled>Click</button>
```

JavaScript

```
JavaScript
let button = document.getElementById("btn");

button.removeAttribute("disabled");
```

Now the **button becomes clickable**.

Example (Complete)

HTML

```
HTML

```

JavaScript

```
JavaScript
let img = document.getElementById("image");

console.log(img.getAttribute("src"));

img.setAttribute("src", "newphoto.jpg");

img.removeAttribute("id");
```

Common HTML Attributes

Attribute	Use
id	Unique element identifier
class	Group of elements
src	Image source
href	Link destination
alt	Image description

Important Points

1. `getAttribute()` → gets attribute value.
2. `setAttribute()` → adds or changes attribute.
3. `removeAttribute()` → removes attribute.

Interview Question

Q: What is attribute manipulation in DOM?

Answer:

Attribute manipulation means **accessing, modifying, adding, or removing HTML element attributes using JavaScript DOM methods like `getAttribute()`, `setAttribute()`, and `removeAttribute()`.**

If you want, the **next very important DOM topic** is:

- **classList (add, remove, toggle)**
- **Navigation in DOM**
- **Creating and Removing Elements**
- **DOM Events**

These are **very important for interviews and projects.** 🚀

You said:

manipulating style (with style attributes)

ChatGPT said:

Manipulating Style in DOM (Using Style Attribute)

In JavaScript DOM, we can **change the CSS style of HTML elements** using the **style property**.

This is called **style manipulation**.

It allows JavaScript to **change colors, sizes, background, fonts, etc. dynamically**.

Definition

Style manipulation means changing the CSS properties of HTML elements using JavaScript DOM.

Syntax

```
JavaScript  
element.style.property = "value";
```

- **element** → selected HTML element
- **style** → property used to access CSS

- **property** → CSS property
- **value** → style value

Example 1: Changing Text Color

HTML

```
HTML
<p id="text">Hello World</p>
```

JavaScript

```
JavaScript
let para = document.getElementById("text");

para.style.color = "red";
```

Output → The text becomes **red**.

Example 2: Changing Background Color

```
JavaScript
let para = document.getElementById("text");

para.style.backgroundColor = "yellow";
```

Example 3: Changing Font Size

```
JavaScript
let para = document.getElementById("text");

para.style.fontSize = "30px";
```

Example 4: Changing Multiple Styles

```
JavaScript
let para = document.getElementById("text");

para.style.color = "blue";
para.style.backgroundColor = "lightgray";
para.style.fontSize = "25px";
```

Important Rule

In JavaScript, **CSS properties use camelCase**.

CSS Property	JavaScript Style Property
background-color	backgroundColor
font-size	fontSize
text-align	textAlign
margin-left	marginLeft

Example:

```
JavaScript
element.style.backgroundColor = "green";
```

Example with Button

HTML

```
HTML
<button onclick="changeStyle()">Click</button>

<p id="demo">Hello JavaScript</p>
```

JavaScript

```
JavaScript
function changeStyle() {
  let text = document.getElementById("demo");

  text.style.color = "white";
  text.style.backgroundColor = "black";
  text.style.fontSize = "28px";
}
```

When the button is clicked → **style changes**.

Common Style Properties

Property	Use
color	text color
backgroundColor	background color
fontSize	text size
textAlign	text alignment
margin	space outside element
padding	space inside element

Important Points

1. Use **element.style.property** to change CSS.
2. CSS properties become **camelCase** in JavaScript.
3. Styles can be changed **dynamically with events**.

Interview Question

Q: How do you change the style of an element using DOM?

Answer:

We can change the style of an HTML element using the **style property**.

Example:

```
JavaScript
document.getElementById("demo").style.color = "red";
```

If you want, the **next very important DOM topics** are:

- **classList (add, remove, toggle)** ★
- **Navigation in DOM (parent, child, sibling)**
- **Creating Elements (createElement)**
- **Adding & Removing Elements (appendChild, remove)**

These are **very important for DOM manipulation and interviews**. 🚀

You said:

classList property all concept

ChatGPT said:

classList Property in DOM (All Concepts)

The **classList** property in JavaScript DOM is used to **add, remove, check, or toggle CSS classes of an HTML element**.

It makes **class manipulation easier** compared to using `setAttribute()`.

1. What is **classList**?

Definition:

classList is a DOM property that returns a **list of all classes of an element** and allows us to manipulate them.

Example HTML:

```
HTML
<p id="text" class="red bold">Hello</p>
```

JavaScript:

```
JavaScript
let element = document.getElementById("text");
console.log(element.classList);
```

Output:

```
["red", "bold"]
```

2. classList Methods

Important methods:

1. `add()`
2. `remove()`
3. `toggle()`
4. `contains()`

3. classList.add()

Definition

Used to **add a new class to an element**.

Syntax

```
JavaScript  
element.classList.add("className");
```

Example

HTML

```
HTML  
<p id="demo">Hello</p>
```

JavaScript

```
JavaScript  
let para = document.getElementById("demo");  
  
para.classList.add("highlight");
```

Now the element will have:

```
class="highlight"
```

4. `classList.remove()`

Definition

Used to **remove a class from an element**.

Syntax

```
JavaScript  
element.classList.remove("className");
```

Example

```
JavaScript  
para.classList.remove("highlight");
```

The class **highlight** is removed.

5. `classList.toggle()`

Definition

`toggle()` **adds the class if it does not exist** and **removes it if it already exists**.

Syntax

```
JavaScript  
element.classList.toggle("className");
```

Example

```
JavaScript  
para.classList.toggle("active");
```

If **active** exists → removed

If **active** does not exist → added

6. classList.contains()

Definition

Used to **check if an element contains a specific class**.

Syntax

```
JavaScript  
element.classList.contains("className");
```

Example

```
JavaScript  
let result = para.classList.contains("active");  
  
console.log(result);
```

Output:

```
true / false
```

7. Complete Example

HTML

```
HTML  
<button onclick="changeStyle()">Click</button>  
  
<p id="text">Hello World</p>
```

CSS

```
CSS  
.highlight{  
  color: white;  
  background-color: black;  
  font-size: 25px;  
}
```

JavaScript

```
JavaScript
function changeStyle(){
  let element = document.getElementById("text");

  element.classList.toggle("highlight");
}
```

Click button → class **highlight** added/removed.

8. Multiple Classes

We can add multiple classes.

```
JavaScript
element.classList.add("class1", "class2");
```

Remove multiple classes:

```
JavaScript
element.classList.remove("class1", "class2");
```

9. Why classList is Better

Instead of this:

```
JavaScript
element.setAttribute("class", "box");
```

We use:

```
JavaScript
element.classList.add("box");
```

It is **cleaner and easier**.

10. className vs classList

className	classList
Changes whole class	Manipulates individual classes
Less flexible	More flexible
Old method	Modern method

Interview Question

Q: What is `classList` in JavaScript?

Answer:

`classList` is a DOM property that allows us to **add, remove, toggle, and check CSS classes of an HTML element.**

If you want, the **next important DOM concepts** are:

- **Navigation in DOM (parent, children, siblings)**
- **Adding Elements (createElement)**
- **appendChild() / append() / prepend()**
- **Removing Elements (remove)**
- **DOM Events (click, submit, input)**

These are **very important for projects and interviews.** 🚀

You said:

navigation on page

ChatGPT said:

DOM Navigation on Page

DOM Navigation means **moving through HTML elements in the DOM tree** using JavaScript.

With DOM navigation, we can access:

- **Parent elements**
- **Child elements**

- **Sibling elements**

This helps JavaScript **move from one element to another element** in the webpage structure.

Example HTML Structure

```
HTML
<div id="parent">
  <h1>Heading</h1>
  <p>Paragraph 1</p>
  <p>Paragraph 2</p>
</div>
```

DOM Tree:

```
div
├── h1
├── p
└── p
```

Here:

- **div** → parent
- **h1** and **p** → children
- two **p** → siblings

1. parentElement

Definition

Used to **get the parent element of an element**.

Syntax

```
JavaScript
element.parentElement
```

Example

```
JavaScript
let element = document.querySelector("p");

console.log(element.parentElement);
```

Output → `<div id="parent">`

2. children

Definition

Returns **all child elements of an element**.

Syntax

```
JavaScript
element.children
```

Example

```
JavaScript
let parent = document.getElementById("parent");

console.log(parent.children);
```

Output → All child elements (`h1`, `p`, `p`).

3. firstElementChild

Definition

Returns the **first child element**.

Syntax

```
JavaScript
element.firstElementChild
```

Example

```
JavaScript
let parent = document.getElementById("parent");

console.log(parent.firstChild);
```

Output → `<h1>`

4. lastElementChild

Definition

Returns the **last child element**.

Syntax

```
JavaScript
element.lastElementChild
```

Example

```
JavaScript
console.log(parent.lastElementChild);
```

Output → `<p>Paragraph 2</p>`

5. nextElementSibling

Definition

Returns the **next sibling element**.

Syntax

```
JavaScript
element.nextElementSibling
```

Example

```
JavaScript
let firstPara = document.querySelector("p");

console.log(firstPara.nextElementSibling);
```

Output → `<p>Paragraph 2</p>`

6. previousElementSibling

Definition

Returns the **previous sibling element**.

Syntax

```
JavaScript
element.previousElementSibling
```

Example

```
JavaScript
let secondPara = document.querySelectorAll("p")[1];

console.log(secondPara.previousElementSibling);
```

Output → `<p>Paragraph 1</p>`

Summary Table

Property	Description
parentElement	Returns parent element
children	Returns all child elements
firstElementChild	Returns first child
lastElementChild	Returns last child
nextElementSibling	Returns next sibling

Property	Description
previousElementSibling	Returns previous sibling

Interview Question

Q: What is DOM navigation?

Answer:

DOM navigation is the process of **accessing and moving between elements in the DOM tree using properties like parentElement, children, nextElementSibling, and previousElementSibling.**

If you want, the **next important DOM topics** are:

- **Adding Elements (createElement)**
- **appendChild() / append() / prepend()**
- **Removing Elements (remove)**
- **DOM Events (click, submit, input)**

These are **very important for building dynamic websites.** 🚀

You said:

adding elements on page

ChatGPT said:

Adding Elements on Page in DOM

In JavaScript DOM, we can **create new HTML elements and add them to the webpage dynamically.**

This process is called **adding elements to the page.**

Main methods used:

1. `createElement()`
2. `appendChild()`
3. `append()`

4. `prepend()`

1. `createElement()`

Definition

`createElement()` is used to **create a new HTML element**.

Syntax

```
JavaScript
document.createElement("tagName");
```

Example

```
JavaScript
let newPara = document.createElement("p");
```

This creates a **new paragraph element**, but it is **not yet added to the page**.

2. `appendChild()`

Definition

`appendChild()` is used to **add a new element inside a parent element at the end**.

Example

HTML

```
HTML
<div id="box"></div>
```

JavaScript

```
JavaScript
let div = document.getElementById("box");

let para = document.createElement("p");

para.innerText = "Hello JavaScript";

div.appendChild(para);
```

Output:

```
HTML
<div id="box">
<p>Hello JavaScript</p>
</div>
```

The paragraph is **added inside the div**.

3. append()

Definition

append() is used to **add elements or text at the end of a parent element**.

Example

```
JavaScript
let div = document.getElementById("box");

let heading = document.createElement("h2");

heading.innerText = "New Heading";

div.append(heading);
```

4. prepend()

Definition

prepend() adds an element **at the beginning of the parent element**.

Example

```
JavaScript
let div = document.getElementById("box");

let heading = document.createElement("h2");

heading.innerText = "First Heading";

div.prepend(heading);
```

The element is added **at the top**.

Example (Complete)

HTML

```
HTML
<div id="container"></div>
```

JavaScript

```
JavaScript
let container = document.getElementById("container");

let newElement = document.createElement("p");

newElement.innerText = "This is a new paragraph";

container.appendChild(newElement);
```

Difference Between appendChild, append, prepend

Method	Position
appendChild()	Adds element at the end
append()	Adds element/text at the end
prepend()	Adds element at the beginning

Important Points

1. `createElement()` creates a new element.
2. `appendChild()` adds element to the DOM.
3. `append()` can add **text and elements**.
4. `prepend()` adds element **at the start**.

Interview Question

Q: How do you add a new element to the webpage using DOM?

Answer:

1. Create element using `createElement()`.
2. Add content using `innerText` or `innerHTML`.
3. Insert it using `appendChild()` or `append()`.

Example:

```
JavaScript
let p = document.createElement("p");
p.innerText = "Hello";
document.body.appendChild(p);
```

If you want, I can explain the **next important DOM topics**:

- **remove() and removeChild() (Removing Elements)**
- **DOM Events (click, submit, input)**
- **Event Listeners**
- **Mini DOM Project (very important for MERN students) 🚀**

You said:

removing elements from page

ChatGPT said:

Removing Elements from Page in DOM

In JavaScript DOM, we can **delete or remove HTML elements from a webpage dynamically**.

This process is called **removing elements from the page**.

Main methods used:

1. `remove()`
2. `removeChild()`

1. remove()

Definition

`remove()` is used to **directly remove an element from the DOM**.

Syntax

```
JavaScript  
element.remove();
```

Example

HTML

```
HTML  
<p id="text">Hello World</p>
```

JavaScript

```
JavaScript  
let element = document.getElementById("text");  
  
element.remove();
```

Output → The paragraph **disappears from the page**.

2. removeChild()

Definition

`removeChild()` is used to **remove a child element from its parent element**.

Syntax

```
JavaScript  
parent.removeChild(child);
```

Example

HTML

```
HTML  
<div id="box">  
  <p id="para">Hello JavaScript</p>  
</div>
```

JavaScript

```
JavaScript  
let parent = document.getElementById("box");  
let child = document.getElementById("para");  
  
parent.removeChild(child);
```

Output → The paragraph is **removed from the div**.

Example with Button

HTML

```
HTML  
<button onclick="removePara()">Remove</button>  
  
<p id="demo">This paragraph will be removed</p>
```

JavaScript

```
JavaScript
function removePara() {
    let para = document.getElementById("demo");

    para.remove();
}
```

Click button → paragraph **disappears**.

Difference Between remove() and removeChild()

Method	Description
remove()	Removes the element directly
removeChild()	Parent removes its child element

Important Points

1. **remove()** is the **simplest way** to delete an element.
2. **removeChild()** requires **parent and child elements**.
3. Both methods remove elements from the **DOM tree**.

Interview Question

Q: How do you remove an element from the DOM?

Answer:

We can remove an element using the **remove()** method.

Example:

```
JavaScript
document.getElementById("demo").remove();
```

If you want, the **next very important DOM concept** is:

DOM Events

- **onclick**

- `addEventListener()`
- `submit`
- `input`
- `mouseover`

These are **very important for building interactive websites and MERN projects**.

You said:

practice

ChatGPT said:

Here are some **DOM Practice Questions** from **basic** → **intermediate** so you can practice everything you learned (selectors, classList, style, navigation, add/remove elements). 

1 Select Element by ID

Task

Change the text of the heading.

HTML

```
HTML
<h1 id="title">Hello</h1>
```

Expected Output

```
Welcome to DOM
```

JavaScript (Practice)

```
JavaScript
let heading = document.getElementById("title");
heading.innerText = "Welcome to DOM";
```

2 Select Element by Class

Task

Change the color of all paragraphs to **blue**.

HTML

```
HTML
<p class="text">Paragraph 1</p>
<p class="text">Paragraph 2</p>
<p class="text">Paragraph 3</p>
```

JavaScript

```
JavaScript
let elements = document.getElementsByClassName("text");

for(let i = 0; i < elements.length; i++){
    elements[i].style.color = "blue";
}
```

3 Using querySelector

Task

Change background color of the first paragraph.

HTML

```
HTML
<p>Hello</p>
<p>Welcome</p>
```

JavaScript

```
JavaScript
let para = document.querySelector("p");

para.style.backgroundColor = "yellow";
```

4 Add Class Using classList

Task

Add class **highlight** to a paragraph.

HTML

```
HTML
<p id="demo">Hello World</p>
```

JavaScript

```
JavaScript
let element = document.getElementById("demo");

element.classList.add("highlight");
```

5 Toggle Class

HTML

```
HTML
<button onclick="change()">Click</button>
<p id="text">Hello</p>
```

JavaScript

```
JavaScript
function change(){
    let element = document.getElementById("text");
    element.classList.toggle("active");
}
```

6 Create New Element

Task

Add a new paragraph to a div.

HTML

```
HTML
<div id="box"></div>
```

JavaScript

```
JavaScript
let div = document.getElementById("box");

let para = document.createElement("p");

para.innerText = "New Paragraph";

div.appendChild(para);
```

7 Remove Element

HTML

```
HTML
<p id="remove">Delete me</p>
<button onclick="deleteText()">Remove</button>
```

JavaScript

```
JavaScript
function deleteText(){
    let element = document.getElementById("remove");
    element.remove();
}
```

8 Navigation Practice

HTML

```
HTML
<div id="parent">
  <p>First</p>
  <p>Second</p>
</div>
```

JavaScript

```
JavaScript
let parent = document.getElementById("parent");

console.log(parent.children);
console.log(parent.firstElementChild);
console.log(parent.lastElementChild);
```

9 Change Multiple Styles

HTML

```
HTML
<p id="text">Hello</p>
```

JavaScript

```
JavaScript
let element = document.getElementById("text");

element.style.color = "white";
element.style.backgroundColor = "black";
element.style.fontSize = "25px";
```

10 Mini Challenge (Important)

Task

When the button is clicked:

- Add a new **list item**

- Text should be **"New Item"**

HTML

```
HTML
<ul id="list">
  <li>Item 1</li>
  <li>Item 2</li>
</ul>

<button onclick="addItem()">Add Item</button>
```

JavaScript

```
JavaScript
function addItem() {
  let ul = document.getElementById("list");

  let li = document.createElement("li");

  li.innerText = "New Item";

  ul.appendChild(li);
}
```



Best DOM Practice Project (Very Important)

If you want, I can give you **3 real DOM projects** that will make your **JavaScript very strong**:

- 1 To-Do List App**
- 2 Color Changer App**
- 3 Dynamic Shopping Cart**

These are **very important for MERN interviews**.

If you want, I can also give you **20 DOM interview questions with answers**.

You said:

Manipulating Style Bing classList obj.classList.classList.add() to add new classes
classList.remove() to remove classes classList.contains() to check if class exists
classList.toggle() to toggle between add & remove

ChatGPT said:

Manipulating Style Using `classList` in DOM

In JavaScript DOM, we can **manipulate (add, remove, check, or toggle) CSS classes of an element** using the `classList` property.

Using `classList` is a **clean and modern way** to change styles instead of directly using `style`.

1. `obj.classList`

Definition

`classList` is a **property that returns a list of all classes of an element**.

Example

HTML

```
HTML
<p id="text" class="red bold">Hello</p>
```

JavaScript

```
JavaScript
let obj = document.getElementById("text");

console.log(obj.classList);
```

Output

```
["red", "bold"]
```

2. `classList.add()`

Definition

`classList.add()` is used to **add a new class to an element**.

Syntax

```
JavaScript  
obj.classList.add("className");
```

Example

```
HTML  
<p id="demo">Hello</p>
```

```
JavaScript  
let obj = document.getElementById("demo");  
  
obj.classList.add("highlight");
```

Now the element becomes:

```
HTML  
<p id="demo" class="highlight">Hello</p>
```

3. `classList.remove()`

Definition

`classList.remove()` is used to **remove a class from an element**.

Syntax

```
JavaScript  
obj.classList.remove("className");
```

Example

```
JavaScript  
obj.classList.remove("highlight");
```

The **highlight class is removed**.

4. `classList.contains()`

Definition

`classList.contains()` checks **whether a class exists in the element or not**.

Syntax

```
JavaScript  
obj.classList.contains("className");
```

Example

```
JavaScript  
let result = obj.classList.contains("active");  
  
console.log(result);
```

Output

```
true or false
```

5. `classList.toggle()`

Definition

`toggle()` **adds a class if it doesn't exist and removes it if it exists**.

Syntax

```
JavaScript  
obj.classList.toggle("className");
```

Example

```
JavaScript  
obj.classList.toggle("dark");
```

If `dark` exists → removed

If `dark` does not exist → added

Complete Example

HTML

```
HTML
<button onclick="change()">Toggle Style</button>

<p id="text">Hello JavaScript</p>
```

CSS

```
CSS
.active{
color:white;
background:black;
font-size:25px;
}
```

JavaScript

```
JavaScript
function change(){
let obj = document.getElementById("text");

obj.classList.toggle("active");
}
```

Click button → **style toggles on/off**.

Summary Table

Method	Purpose
<code>obj.classList</code>	Returns list of classes
<code>classList.add()</code>	Adds new class
<code>classList.remove()</code>	Removes class
<code>classList.contains()</code>	Checks class existence
<code>classList.toggle()</code>	Adds/removes class automatically

Interview Definition

classList is a DOM property used to add, remove, check, or toggle CSS classes of an HTML element.

If you want, I can also explain the **next DOM topic that usually comes after this**:

🔥 DOM Events (Very Important)

- `onclick`
- `addEventListener()`
- `input`
- `submit`
- `mouseover`

These are **used in almost every JavaScript project**.

You said:

Adding Elements `uocument.createElement('p') • appendChild( element) • append( element) • prepend( element) • insertAdjacent( where, element)`

ChatGPT said:

Adding Elements in DOM

In JavaScript DOM, we can **create new HTML elements and insert them into the webpage dynamically**.

This is called **adding elements to the page**.

Main methods used:

- `document.createElement()`
- `appendChild()`
- `append()`
- `prepend()`

- `insertAdjacentElement()` / `insertAdjacentHTML()`

1. `document.createElement()`

Definition

`createElement()` is used to **create a new HTML element**.

Syntax

```
JavaScript
document.createElement("tagName");
```

Example

```
JavaScript
let para = document.createElement("p");
```

This creates a **new paragraph element**, but it is **not yet visible on the page**.

2. `appendChild()`

Definition

`appendChild()` is used to **add a new element at the end of a parent element**.

Example

HTML

```
HTML
<div id="box"></div>
```

JavaScript

```
JavaScript
let div = document.getElementById("box");

let para = document.createElement("p");

para.innerText = "Hello JavaScript";

div.appendChild(para);
```

Output

```
HTML
<div id="box">
<p>Hello JavaScript</p>
</div>
```

3. append()

Definition

append() adds **elements or text at the end of a parent element**.

Example

```
JavaScript
let div = document.getElementById("box");

let heading = document.createElement("h2");

heading.innerText = "New Heading";

div.append(heading);
```

Difference:

append() can also add **text directly**.

Example

```
JavaScript
div.append("Hello");
```

4. prepend()

Definition

`prepend()` adds an element **at the beginning of a parent element**.

Example

```
JavaScript
let div = document.getElementById("box");

let heading = document.createElement("h2");

heading.innerText = "First Heading";

div.prepend(heading);
```

Now the heading appears **before other elements**.

5. insertAdjacentElement()

Definition

`insertAdjacentElement()` inserts an element **relative to another element at a specific position**.

Syntax

```
JavaScript
element.insertAdjacentElement(position, newElement);
```

Positions

Position	Meaning
----------	---------

<code>beforebegin</code>	Before the element
--------------------------	--------------------

<code>afterbegin</code>	Inside element at start
-------------------------	-------------------------

<code>beforeend</code>	Inside element at end
------------------------	-----------------------

<code>afterend</code>	After the element
-----------------------	-------------------

Example

HTML

```
HTML
<div id="box">
  <p>Hello</p>
</div>
```

JavaScript

```
JavaScript
let box = document.getElementById("box");

let newPara = document.createElement("p");

newPara.innerText = "New Paragraph";

box.insertAdjacentElement("beforeend", newPara);
```

Summary Table

Method	Use
<code>createElement()</code>	Creates new element
<code>appendChild()</code>	Adds element at end
<code>append()</code>	Adds element/text at end
<code>prepend()</code>	Adds element at beginning
<code>insertAdjacentElement()</code>	Inserts element at specific position

Interview Question

Q: How do you create and add a new element in DOM?

Answer:

1. Create element using `createElement()`
2. Add content using `innerText`

3. Insert it using `appendChild()` or `append()`

Example:

```
JavaScript
let p = document.createElement("p");
p.innerText = "Hello";
document.body.appendChild(p);
```

If you want, I can also explain the **last important DOM topic**:

Removing Elements

- `remove()`
- `removeChild()`

And after that I can give you **a full DOM cheat sheet for interviews**.

You said:

Practice Qs Id the following elements to the container using only JavaScript and the DOM methods. i a `<p>` with red text that says "Hey I'm red!" ii an `<h3>` with blue text that says "I'm a blue h3!" iii a `<div>` with a black border and pink background color with the following elements inside of it: • another `<h1>` that says "I'm in a div" • a `<p>` that says "ME TOO!"

ChatGPT said:

Below is a **DOM Practice Solution** where we **create and add elements using only JavaScript**.

HTML (Container)

```
HTML
<div id="container"></div>
```

This **container** is where all elements will be added.

JavaScript Solution

```
JavaScript
// select container
let container = document.getElementById("container");

// i) Create <p> with red text
let para = document.createElement("p");
para.innerText = "Hey I'm red!";
para.style.color = "red";

container.appendChild(para);

// ii) Create <h3> with blue text
let heading3 = document.createElement("h3");
heading3.innerText = "I'm a blue h3!";
heading3.style.color = "blue";

container.appendChild(heading3);

// iii) Create div with border and pink background
let newDiv = document.createElement("div");
newDiv.style.border = "2px solid black";
newDiv.style.backgroundColor = "pink";

// Create <h1>
let heading1 = document.createElement("h1");
heading1.innerText = "I'm in a div";

// Create <p>
let para2 = document.createElement("p");
para2.innerText = "ME TOO!";

// add h1 and p inside div
newDiv.appendChild(heading1);
newDiv.appendChild(para2);

// add div to container
container.appendChild(newDiv);
```

Concepts Used in This Practice

- `document.createElement()`
- `appendChild()`
- `style`
- DOM element selection (`getElementById`)

- `classList`
- `querySelector`
- `createElement`
- `remove`
- `events`

ChatGPT can make mistakes. Check important info. See [Cookie Preferences](#).